

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint Based Problem Solving

Course Code:	ITA0512	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	S. Susannal Devapriyam	Reg. No.:	192525398
Section / Batch:	B. Tech AI & ML	Date:	26 - 08 - 2026

Code of Conduct

I, S. Susannal Devapriyam (192525398) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Susannal Devapriyam

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

CASE STUDY DOCUMENT

Document Scanner Application Using OpenCV: Pipeline Design and Preprocessing for Improved Document Quality

1. Introduction

Mobile document scanner applications have become widely useful in education, offices, banking, business, healthcare, and personal document management. Unlike traditional flatbed scanners, mobile scanners use a smartphone camera to capture a physical document and convert it into a clean digital representation.

However, directly capturing a document using a mobile camera does not always produce a high-quality scanned image. The captured image may contain several problems such as:

- Uneven lighting and shadows.
- Perspective distortion due to camera angle.
- Background objects around the document.
- Low contrast between text and paper.
- Camera noise and blur.
- Skewed or rotated documents.
- Unwanted colours and illumination variations.

Therefore, a mobile document scanner requires an effective **Computer Vision and image preprocessing pipeline** to transform a raw camera image into a clean, readable, and properly aligned scanned document.

The main requirements given in the problem are:

- A. Capture document images
- B. Enhance readability
- C. Output clean scanned images

The proposed solution uses OpenCV-based image processing techniques such as grayscale conversion, noise reduction, edge detection, contour detection, perspective transformation, contrast enhancement, thresholding, and morphological processing.

2. Problem Understanding

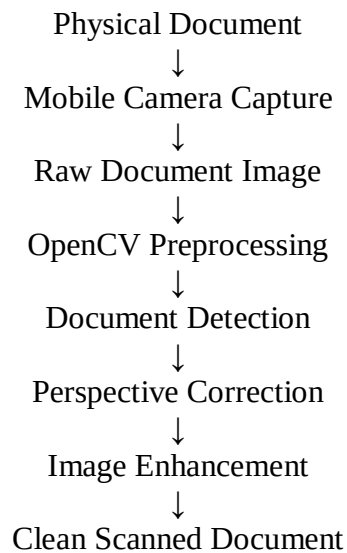
2.1 Problem Definition

The problem involves designing a mobile application that captures an image of a physical document using a smartphone camera and processes it to generate a clean scanned version.

The input image is not guaranteed to be perfect because it is captured under real-world conditions. For example, a user may hold the camera at an angle, capture the image under poor lighting, or place the document on a cluttered background.

Therefore, the system must automatically identify the document region and improve its visual quality.

The overall transformation can be represented as:



The major challenge is not simply capturing an image. The system must intelligently process the image so that the final output resembles a scanned document.

2.2 Input Requirements

The input to the system is an image captured using a mobile camera.

The captured image may contain:

- A document placed on a table.

- Background objects surrounding the document.
- Shadows caused by lighting.
- Perspective distortion.
- Uneven brightness.
- Noise from the camera sensor.
- Slight motion blur.
- Different document sizes.

Therefore, the input image must undergo preprocessing before it can be considered a clean digital document.

2.3 Output Requirements

The final output should satisfy the following characteristics:

1. The document boundaries should be properly identified.
2. The document should be correctly aligned.
3. Perspective distortion should be removed.
4. Text should be clearly visible and readable.
5. Background noise should be minimized.
6. Shadows and uneven illumination should be reduced.
7. The final image should resemble a scanned document.

Thus, the expected output is not simply an enhanced photograph. It should be a clean, readable, geometrically corrected document image.

2.4 Major Constraints

The system must operate under several practical constraints.

Constraint 1: Variable Lighting

Mobile documents may be captured in bright, dark, or uneven lighting conditions.

Constraint 2: Perspective Distortion

The camera may not be perfectly parallel to the document.

Constraint 3: Background Interference

Objects around the document may affect edge detection.

Constraint 4: Processing Efficiency

A mobile application should process images within a reasonable time.

Constraint 5: Text Preservation

Preprocessing should improve readability without removing important text details.

Constraint 6: Image Noise

Camera sensors may introduce noise, especially in low-light conditions.

3. Constraint Analysis

3.1 Importance of Constraint Analysis

Before designing the solution, it is important to understand which problems have the greatest impact on document quality.

A document scanner must handle both geometric problems and image quality problems.

Geometric problems include:

- Rotation.
- Skew.
- Perspective distortion.

Image quality problems include:

- Noise.
- Shadows.
- Low contrast.
- Uneven brightness.
- Blurred text.

These problems require different preprocessing methods.

3.2 Constraint Prioritization

The constraints can be prioritized as follows:

Priority	Constraint	Impact
High	Correct document detection	Determines the document region
High	Perspective correction	Produces proper document alignment
High	Text readability	Ensures useful output
Medium	Noise reduction	Improves visual quality
Medium	Lighting correction	Reduces shadows and uneven brightness
Medium	Processing speed	Important for mobile applications
Low	Colour preservation	Optional for text documents

The highest priority is to correctly detect and align the document. Without proper document detection, later enhancement steps cannot produce a correct result.

3.3 Analysis of Lighting Problems

Lighting is one of the major challenges in mobile document scanning.

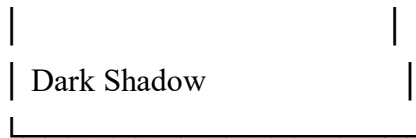
Consider a document captured under a single overhead light source. One side of the paper may appear bright while another side appears dark.

This causes problems because global thresholding may incorrectly classify dark paper regions as text.

For example:

Original Document

Bright Area	
TEXT	



To solve this problem, adaptive image enhancement techniques can be applied.

Adaptive thresholding is particularly useful because it calculates threshold values based on local image regions rather than using one global value.

3.4 Analysis of Perspective Distortion

Perspective distortion occurs when the camera captures the document from an angle.

The captured document appears trapezoidal instead of rectangular.

This problem can be solved using a **perspective transformation**.

The four corners of the detected document are mapped to a rectangular output image.

3.5 Analysis of Noise

Camera images may contain unwanted pixel variations.

Noise can occur because of:

- Low-light camera operation.
- Sensor limitations.
- Image compression.
- Transmission errors.

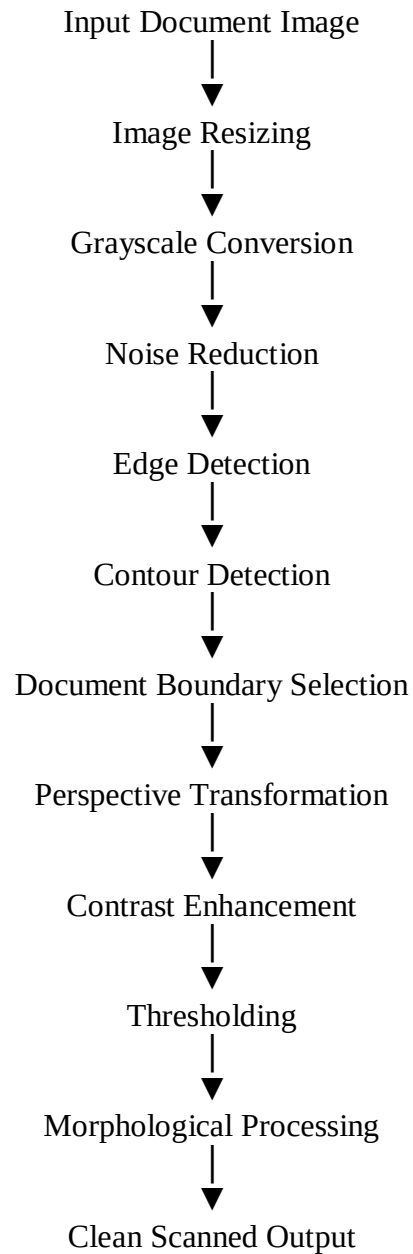
Noise can reduce the accuracy of edge detection and thresholding.

Therefore, smoothing techniques such as **Gaussian Blur** can be used before edge detection.

However, excessive smoothing should be avoided because it can remove small text details.

4. Proposed OpenCV Pipeline

The proposed solution uses a multi-stage OpenCV image processing pipeline.



Each stage performs a specific task and contributes to improving the overall document quality.

5. Stage 1 – Image Acquisition

The first stage involves capturing the document image using a mobile camera.

The image can be obtained through:

- Smartphone camera.
- Uploaded image.
- Real-time video frame.

The raw captured image is represented as:

$I(x,y)$

where:

- x represents the horizontal pixel location.
- y represents the vertical pixel location.
- $I(x,y)$ represents the intensity or colour value.

The quality of image acquisition directly influences the performance of all later stages.

A highly blurred image may not be fully recoverable using preprocessing techniques.

Therefore, the application should ideally guide the user to hold the camera steady and ensure the document is visible.

6. Stage 2 – Image Resizing

Mobile cameras can capture very high-resolution images.

Processing the full-resolution image can increase computational cost.

Therefore, the image can be resized to a suitable working resolution.

Example:

```
image = cv2.resize(image, (800, 600))
```

Resizing improves processing speed and reduces memory consumption.

However, excessive resizing should be avoided because small text and document edges may lose important details.

Therefore, a balanced resolution should be selected.

7. Stage 3 – Grayscale Conversion

The captured document image is generally represented using three colour channels:

- Red.
- Green.
- Blue.

For document edge detection, colour information is often unnecessary.

Therefore, the image is converted into grayscale.

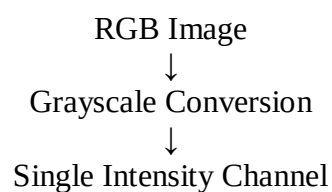
```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

The grayscale image contains only intensity information.

This conversion provides several advantages:

1. Reduces computational complexity.
2. Simplifies image processing.
3. Makes edge detection easier.
4. Removes unnecessary colour variations.
5. Prepares the image for thresholding.

The conversion can conceptually be represented as:



8. Stage 4 – Noise Reduction

Before detecting document edges, unwanted noise should be reduced.

A Gaussian Blur can be applied:

```
blurred = cv2.GaussianBlur(gray, (5,5), 0)
```

Gaussian Blur smooths small intensity variations while preserving major structures.

The purpose of this stage is to prevent random noise from producing false edges.

Why Gaussian Blur is Used

Gaussian Blur is suitable because:

- It reduces high-frequency noise.
- It smooths the image gradually.
- It improves the stability of edge detection.
- It is computationally efficient.
- It preserves important document structures better than excessive averaging.

However, the blur kernel should not be too large.

9. Stage 5 – Edge Detection

The next step is to identify strong boundaries in the image.

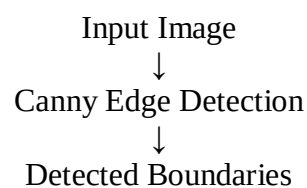
A common OpenCV technique is Canny Edge Detection.

```
edges = cv2.Canny(blurred, 50, 150)
```

Canny Edge Detection identifies regions with significant changes in intensity.

Document boundaries usually produce strong edges because the paper and background often have different intensity values.

The output may appear as:



Importance of Edge Detection

Edge detection helps to:

- Identify document borders.
- Separate the document from the background.
- Locate major structural boundaries.
- Support contour detection.
- Reduce unnecessary image information.

10. Stage 6 – Contour Detection

After edge detection, the system identifies continuous boundaries called contours.

In OpenCV:

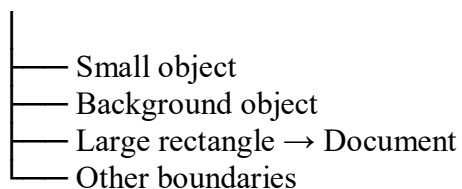
```
contours, hierarchy = cv2.findContours(  
    edges,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE )
```

Contours represent connected boundaries detected in the image.

The document is usually identified as a large rectangular contour.

The contours can be sorted according to their area.

Detected Contours



The system should select the contour most likely to represent the document.

11. Stage 7 – Document Boundary Detection

A document generally contains four major corners.

After contour detection, polygon approximation can be applied.

```

approx = cv2.approxPolyDP(
    contour,
    epsilon,
    True
)

```

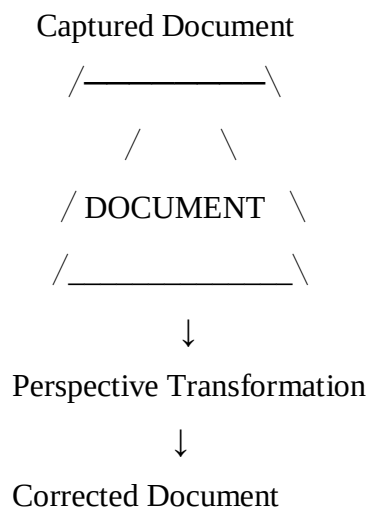
If the detected contour contains approximately four points, it can be considered a potential document boundary.

12. Stage 8 – Perspective Transformation

This is one of the most important stages in a document scanner.

A document captured from an angle may appear distorted.

Perspective transformation maps the detected document corners to a rectangular output.



OpenCV provides:

```
matrix = cv2.getPerspectiveTransform(source, destination)
```

```
warped = cv2.warpPerspective(
    image,

```

```
matrix,  
(width, height)  
)
```

Justification

Perspective transformation is necessary because:

1. Users may capture images from different angles.
2. Documents may appear trapezoidal.
3. Text alignment can become distorted.
4. A rectangular output improves readability.
5. It produces an appearance similar to a traditional scanner.

13. Stage 9 – Contrast Enhancement

After perspective correction, the document may still have poor contrast.

For example:

- Text may appear faded.
- Paper may appear grey.
- Shadows may reduce readability.

Contrast enhancement increases the visual difference between text and background.

A suitable method is **CLAHE (Contrast Limited Adaptive Histogram Equalization)**.

```
clahe = cv2.createCLAHE(  
    clipLimit=2.0,  
    tileGridSize=(8,8)  
)  
enhanced = clahe.apply(gray)
```

CLAHE processes local regions independently.

Advantages of CLAHE

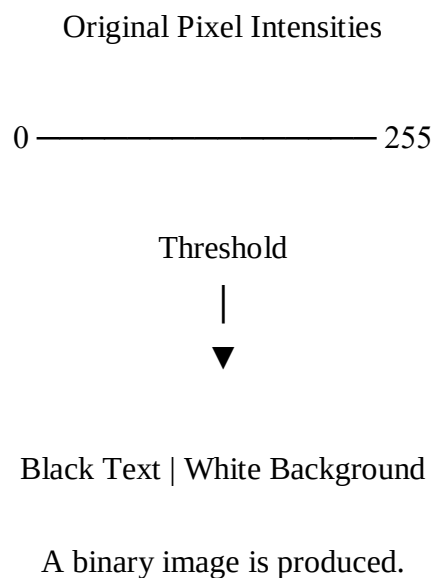
- Handles uneven illumination.
- Improves local contrast.
- Enhances faded text.
- Prevents excessive global contrast amplification.
- Produces better results for documents with shadows.

This makes CLAHE particularly useful for mobile document scanning.

14. Stage 10 – Thresholding

Thresholding converts the image into a binary representation.

For a typical black-and-white document:



Example using adaptive thresholding:

```
binary = cv2.adaptiveThreshold(  
    enhanced,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    11,
```

2

)

Why Adaptive Thresholding is Preferred

Global thresholding uses one threshold value for the entire image.

This can fail when lighting is uneven.

Adaptive thresholding improves document readability under different lighting conditions.

15. Stage 11 – Morphological Processing

After thresholding, small noise or gaps may still remain.

Morphological operations can refine the document image.

Common operations include:

Erosion

Erosion removes small foreground pixels.

Dilation

Dilation expands foreground regions.

Opening

Opening removes small noise.

Closing

Closing fills small gaps.

Example:

```
kernel = np.ones((2,2), np.uint8)
```

```
cleaned = cv2.morphologyEx(  
    binary,
```



```
cv2.MORPH_OPEN,  
kernel  
)
```

The correct morphological operation depends on the document condition.

16. Final Conclusion

The Mobile Document Scanner Application requires more than simply capturing a photograph of a document. The raw image may contain noise, shadows, background objects, perspective distortion, and poor contrast.

A structured OpenCV preprocessing pipeline can effectively address these problems.

The proposed pipeline consists of:

Image Acquisition → Resizing → Grayscale Conversion → Gaussian Blur → Edge Detection → Contour Detection → Document Boundary Detection → Perspective Transformation → Contrast Enhancement → Adaptive Thresholding → Morphological Processing → Clean Output

Each preprocessing technique has a specific role in improving document quality.

- **Gaussian Blur** reduces noise.
- **Canny Edge Detection** identifies document boundaries.
- **Contours** locate the document.
- **Perspective Transformation** corrects camera angle.
- **CLAHE** improves local contrast.
- **Adaptive Thresholding** handles uneven lighting.
- **Morphological Operations** refine the final text.

The solution satisfies all primary requirements of capturing document images, enhancing readability, and generating clean scanned outputs.

Most importantly, the design is based on logical constraint analysis. Instead of applying random image-processing operations, each OpenCV technique is selected and justified according to a specific problem encountered in real-world mobile document scanning.

Therefore, the proposed OpenCV pipeline provides a practical, feasible, and effective solution for developing a mobile document scanner application.